Movie Ticketing System Project

Software Design Specification

Version 2.0

03/05/26

Group #13
Jack Alleva, Andrew Saluta, Gershom Mushagalusa

Prepared for
CS 250- Introduction to Software Systems
Instructor: Dr. Gus Hanna
Spring 2026

Revision History

Date	Description	Author	Comments
2/12/26	Version 1.0	Jack Alleva	First Revision
3/5/26	Version 2.0	Andrew Saluta	Software Design Specification Revision

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Jack Alleva, Andrew Saluta, Gershom Mushagalusa	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

REVISION HISTORY..... I

DOCUMENT APPROVAL..... I

1. INTRODUCTION..... 1

1.1 Purpose..... 1
1.2 Scope..... 1
1.3 Definitions, Acronyms, and Abbreviations..... 1
1.4 References..... 1

1.5 Overview 1

2. GENERAL DESCRIPTION 1

2.1 Product Perspective 1
2.2 Product Functions 1
2.3 User Characteristics 1
2.4 General Constraints 1
2.5 Assumptions and Dependencies 1

3. SPECIFIC REQUIREMENTS 1

3.1 External Interface Requirements 1
3.1.1 User Interfaces 1
3.1.2 Hardware Interfaces 1
3.1.3 Software Interfaces 1
3.1.4 Communications Interfaces 1
3.2 Functional Requirements 1
3.2.1 <Functional Requirement or Feature #1> 1
3.2.2 <Functional Requirement or Feature #2> 1
3.3 Use Cases 1
3.3.1 Use Case #1 1
3.3.2 Use Case #2 1
3.4 Classes / Objects 1
3.4.1 <Class / Object #1> 1
3.4.2 <Class / Object #2> 1
3.5 Non-Functional Requirements 1
3.5.1 Performance 1
3.5.2 Reliability 1
3.5.3 Availability 1
3.5.4 Security 1
3.5.5 Maintainability 1
3.5.6 Portability 1
3.6 Inverse Requirements 1
3.7 Design Constraints 1
3.8 Logical Database Requirements 1
3.9 Other Requirements 1

4. ANALYSIS MODELS 1

4.1 Sequence Diagrams 1
4.3 Data Flow Diagrams (DFD) 1
4.2 State-Transition Diagrams (STD) 1

5. CHANGE MANAGEMENT PROCESS 1

A. APPENDICES 1

A.1 Appendix 1 1
A.2 Appendix 2 1

1. Introduction

The Software Requirements Specification (SRS) for the Ticket Processing Machine (TPM) Movie Ticketing System provides a comprehensive description of the system's objectives, functionality, constraints, and overall structure. This document defines the operational requirements for a secure and efficient movie ticketing platform that supports both online and in-person kiosk purchases. The system is designed to assist movie theaters in managing ticket sales, preventing activity such as bot purchasing and ticket reselling, ensuring accurate seat availability and transaction processing.

1.1 Purpose

The purpose of this SRS is to clearly define and document the requirements of the Ticket Processing Machine (TPM). This document gathers all major system expectations, including user needs, system functionality, and technical constraints, in order to provide a complete understanding of how the system should operate. It establishes a shared reference for the development team and stakeholders to ensure that the final product aligns with the intended goals.

This specification also helps clarify how the system will be used in real world scenarios, such as online ticket purchases and theater kiosk transactions. By outlining both functional and non functional requirements, this document supports informed design decisions and reduces confusion during development. Ultimately, the SRS ensures that the system is built in a structured, organized, and goal-oriented manner.

1.2 Scope

The TPM will provide a secure platform for purchasing movie tickets both online and through in person kiosks. The TPM will retrieve movie schedules directly from participating theaters and display organized listings that include movie titles, ratings, showtimes, and available formats such as standard or IMAX. The system will present and interactive seating layout that allows users to visually select their preferred seats while instantly updating availability to prevent seating conflicts and confusion. The system will allow customers to browse movies, select showtimes, choose seats with real time availability, complete secure payments, and receive QR code tickets for entry.

In addition to handling ticket purchases, the system will reduce bot purchases and reselling by incorporating anti-bot mechanisms such as CAPTCHA verification and linking tickets to verified user identification. It will store transaction records and provide sales data to participating theaters. The TPM is designed to support high user traffic while delivering a reliable and efficient ticket purchasing experience.

-

1.3 Definitions, Acronyms, and Abbreviations

TPM	**Ticket Processing Machine** - The movie ticketing system described in this document that manages online and kiosk ticket purchases
QR Code	**Quick Response Code** - A machine readable code generated by the TPM that stores ticket information and is scanned for theater entry
CAPTCHA	**Completely Automated Public Turing test to tell Computers and Humans Apart** - A security mechanism to prevent automated bot purchases
EMV	**Europay, Mastercard, and Visa -** The original creators of a global security standard for chip-based payment cards, terminals, and automated teller machines (ATMs)
POS	**Point of Sale -** The time and place at which a retail transaction is completed
API	**An Application Programming Interface -** A set of rules and protocols that allows different software applications to communicate, exchanging data and functionality
KYC	**Know Your Customer -** A mandatory, multi-step regulatory process used by financial institutions and businesses to verify a client's identity, assess risks, and prevent fraud, money laundering, and terrorism financing.
TLC	**Transport Layer Security -** A cryptographic protocol designed to provide secure, end-to-end communications over computer networks, such as the internet, by ensuring data privacy, integrity, and authentication
SRS	**Software Requirements Specification** - A document that defines a software system's requirements, functionality, and constraints

1.4 References

1. Non-Fungible Token Standard
   Report number/date: EIP-721 (first published 2018; living EIP page).
   Publishing organization/source: Ethereum.
   Where to obtain: Ethereum Improvement Proposals (EIPs) repository
   https://eips.ethereum.org/EIPS/eip-721

   https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/
   https://www.clickssl.net/blog/tls-versions

1.5 Overview

The remainder of this document provides a detailed description of the TPM system and its requirements. Section 2 presents the overall system description, including product perspective, product functions, user characteristics, constraints, and assumptions. Section 3 defines the specific functional and non-functional requirements, external interface requirements, and database needs. Section 3 also defines precisely what the system must do and the standards it must meet

2. General Description

This section describes the overall context and general factors that influence the design and requirements of the TPM. It provides an understanding of the system's purpose, environment, users, and limitations. The information presented here does not define specific system requirements but instead establishes the background necessary to understand detailed requirements provided in Section 3

2.1 Product Perspective

The TPM is a centralized movie ticketing system designed to support both online customers and in-theater kiosk users. It functions as an independent platform that integrates with theater scheduling systems and external payment processing services. The system gets movie listings, showtimes, and seating information from participating theaters and presents them though an organized user interface.

2.2 Product Functions

At a high level, the TPM enables customers to browse movies, view available showtimes, and select specific seats through seating charts. It supports both online and kiosk based purchases and processes secure payments through payment gateways. The system generates digital tickets containing QR codes for theater entry and maintains records of transactions for tracking and

reporting purposes. It also incorporates protective measures to reduce bot purchases and ticket reselling.

2.3 User Characteristics

The TPM serves three primary user groups: customers, theater staff, and system administrators. Customers are members of the general public who purchase tickets either online or at kiosks. They are expected to have basic familiarity with web interfaces. The system interface must be simple and easy to navigate.
Theater staff interact with the system primarily for ticket validation and sales monitoring. They require a straightforward interface for scanning QR codes and viewing transaction information. System administrators are responsible for maintaining the system, monitoring performance, and managing data. These users are expected to have technical knowledge and familiarity with database and system management tools.

2.4 General Constraints

The design of the TPM is subject to several general constraints. The system must integrate with external payment gateways and theater schedule systems, which may limit certain technical design choices. It must comply with security requirements for handling financial transactions. The system must also operate within budget limitations and available hardware capabilities for kiosk development. Additionally, performance expectations, such as supporting high volumes of users, may influence decisions.

2.5 Assumptions and Dependencies

The TPm assumes that participating theaters will provide accurate and up to date movie schedules, seating layouts, and pricing information. It also assumes that reliable internet connection will be available for both online and theater kiosks. The system depends on the continued availability of third party services such as payment gateways and CAPTCHA verifications systems. Any disruption in these external services may impact system functionality. The TPM also assumes that users will provide accurate identification information when required for ticket verification purposes.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

Web storefront (desktop and mobile access)

- Description: Web UI that displays theaters, movies, showtimes, seat maps, pricing, loyalty features, and purchase flow which is a must priority

- Inputs: User navigation, search/filter preferences, seat selections, payment credentials, and authentication tokens.
- Processing: Query schedule/catalog service, lock seat(s) for the checkout time window, calculate taxes/currency conversion, and present payment options.
- Outputs: Live seat map (based on availability), order confirmation page, QR & blockchain ticket issued.
- Error handling: If seat lock fails, present an atomic failure message with suggested alternative seats or requeue.
- Verification: End-to-end test: search then seat select then pay then receive ticket. Automated UI tests must assert seat-availability changes across concurrent sessions.
- Trace: Use cases: BrowseMovies, ReserveSeats, PurchaseTickets.

Kiosk UI (in-theater)

- Same structure as Web storefront but with simplified flow, accessibility options, local language default, and hardware payment reader integration.
- Verification: Hardware-in-the-loop tests and kiosk acceptance tests.

Admin dashboard

- For theater managers or personnel: show sales, cancel/void transactions, manage showtime listings.
- Security: two-factor admin login and role-based access controls.
- Verification: Role-based tests and audit trail checks.

3.1.2 Hardware Interfaces

Web storefront: Card reader / POS

- Support EMV chip & contactless readers according to the merchant's hardware.
- Verification: Integration tests with certified POI devices.

Kiosk: Kiosk printers / scanners

- Required for receipts/ticket printouts and scanning QR codes at entry points.
- Verification: Acceptance tests on sample hardware.

3.1.3 Software Interfaces

Web storefront: Theater schedule API

- API for real-time seat and showtime data, using push notifications or cached polling, is a must priority
- Verification: contract tests (consumer-driven contract), load tests.

Kiosk: Payment gateway interfaces

- Integrations with payment processors for cards and PayPal, payment processors/clearing services
- Verification: sandbox transactions and PCI scope validation.

Admin dashboard: Identity provider & KYC

- External IdP and optional KYC provider integration for loyalty account verification and anti-resell measures.
- Verification: compliance tests and identity-proofing workflows.

3.1.4 Communications Interfaces

TLS encryption

- All communications (web, APIs, kiosk-backend) must use TLS 1.2 or higher.
- Verification: automated security scan and cert validation.

WebSocket / push

- Use for near-real-time seat availability updates during seat selection and for large-event queue progression notifications.
- Verification: stress tests for WebSocket connections and fallback to long polling.

3.2 Functional Requirements

3.2.1 Browse & listing

3.2.1.1 Introduction: System shall list movies, showtimes, ratings, formats (e.g., standard, IMAX), and theater info.
3.2.1.2 Inputs: query (location, filters, date).
3.2.1.3 Processing: retrieve schedules from the theater API, then apply business rules (age restrictions, format availability).
3.2.1.4 Outputs: Browseable showtime schedules with seat availability.
3.2.1.5 Error handling: present cached availability or “information temporarily unavailable” with retry CTA and is of high priority
Verification: Through API contract tests, UI tests for filtering and sort behavior.

3.2.2 Seat selection & locking

3.2.2.1 Introduction: Users can visually select seats; the system must lock selected seat(s) for a configurable checkout window (e.g., 5 minutes). Only if the lock succeeds may the checkout proceed.
3.2.2.2 Inputs: seat selection, user token (if logged in), and showing id.
3.2.2.3 Processing: atomic seat reservation transaction against the showing. If concurrent attempts occur, seat allocation is first-commit-wins.
3.2.2.4 Outputs: lock confirmation or seat-unavailable error.
3.2.2.5 Error handling: if lock expires or fails, return a specific code and show alternatives or rollback partial locks.
Verification: concurrency tests that simulate simultaneous seat selection across multiple clients; automated tests to ensure no double bookings.

3.2.3 Checkout & payment

3.2.3.1 Introduction: Accept payments via third-party processors. Ensure price consistency across channels and local-currency conversion for display/charging. Single transaction capped at 20 tickets.
3.2.3.2 Inputs: payment method details, billing info, seat lock token.

3.2.3.3 Processing: validate seat-lock, calculate final price, route payment to appropriate gateway, finalize and persist order, then issue ticket.
3.2.3.4 Outputs: payment confirmation, ticket(s) (QR + blockchain record).
3.2.3.5 Error handling: If there's a declined payment display payment errors page, which unlocks seats and prompt retry instructions
Verification: payment gateway sandbox tests, currency conversion unit tests, limit tests (20 tickets), reconciliation tests.

3.2.4 Ticket issuing & uniqueness

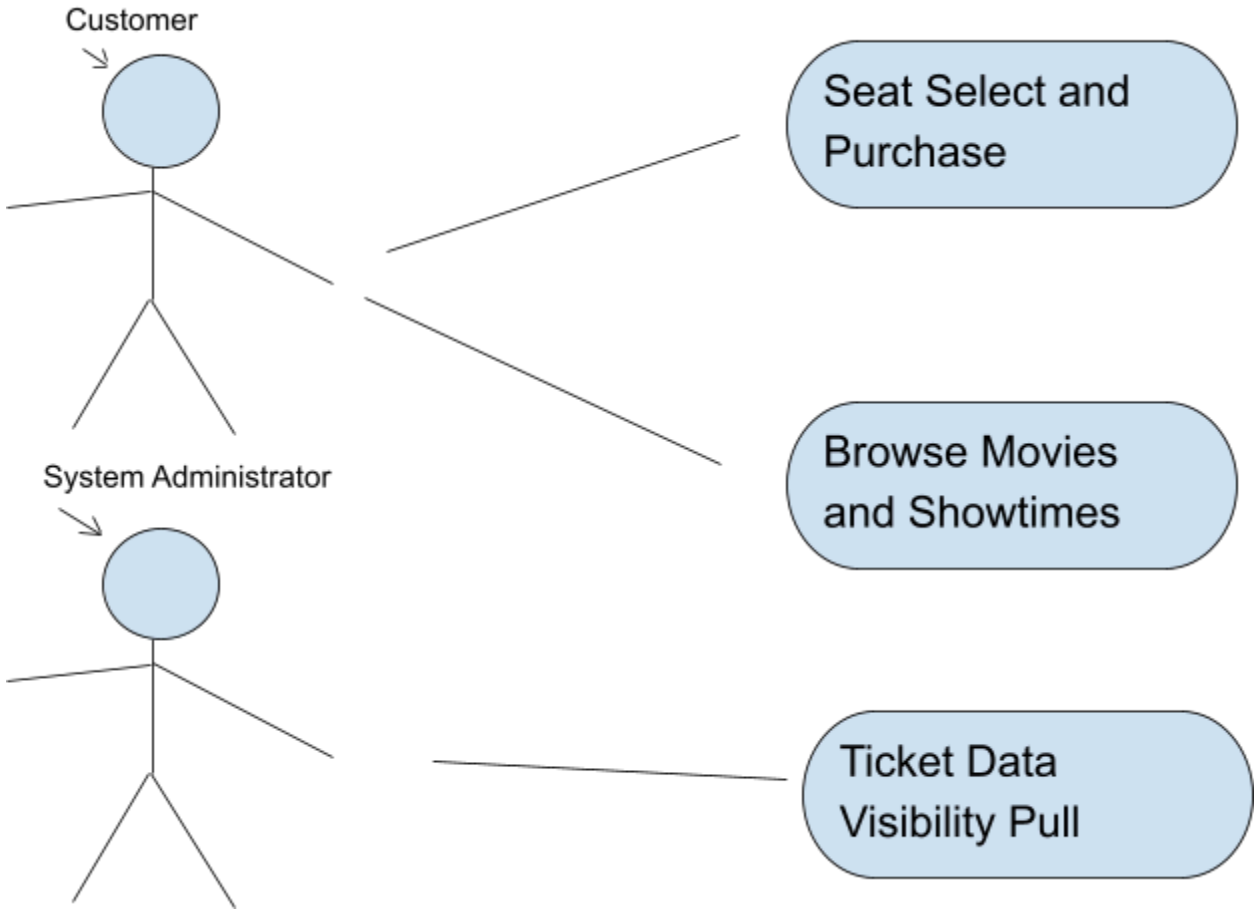
3.2.4.1 Introduction: For each successful purchase the system shall generate a QR-Code ticket or produce a unique on-chain token representing the ticket (NFT) so tickets are unique and non-replicable
3.2.4.2 Input: finalized order data, purchaser id/identifier (if account-based)
3.2.4.3 Processing: create QR payload (signed), mark blockchain NFT token transfer/ownership per the chosen standard
3.2.4.4 Outputs: QR image with an encoded ticket payload or a blockchain token id and transaction receipt
3.2.4.5 Error handling: if blockchain production fails but payment succeeded, place order in pending-production queue and notify purchaser. Then we set clear fallbacks for entry.

3.2.5 Anti-bot & Anti-resell measures

3.2.5.1 Introduction: Prevent automated mass purchases and ticket scalping through layered measures such as CAPTCHA during checkout for suspicious flows, rate-limiting per IP/account, KYC for high-volume buyers, and linking tickets to verified identity for transfers/resale restrictions.
3.2.5.2 Inputs: Log device/network data and analyze purchase behavior for fraud detection.
3.2.5.3 Processing: Evaluate transaction risk and issue verification prompts for flagged accounts
3.2.5.4 Outputs: Allow/deny/challenge responses.
3.2.5.5 Error handling: False-positive flows must offer human review and clearly document the appeal/resolution process.

3.3 Use Cases

Use Case Diagram



3.3.1 Use Case #1

- **Name of Use Case:** Seat Select and Purchase
- Actor(s): Customer
- Flow of events:
    - 1. Customer connects to ticket selling application
    - 2. Customer searches for a movie, selecting by date and location
    - 3. System retrieves schedule and displays available showtimes
    - 4. Customer selects a desired showtime
    - 5. System retrieves interactive seat map with up-to-date selection
    - 5. Customer selects what seats they would like to purchase out of all available seats
    - 6. System locks seats from other customer purchase for 5 minutes
- Entry Conditions:
    - 1. System is connected to theater schedule and seat data API
    - 2. Computing requirements: Compatible browser/app, Internet connection for live updates

3.3.2 Use Case #2

- **Name of Use Case:** Browse Movies and Showtimes
- Actor(s): Customer
- Flow of events:
    - 1. Customer connects to ticket selling application
    - 2. Customer enters movie browsing tab
    - 3. System retrieves movies, separated by currently showing and showing soon
    - 4. Customer browses through list of movies and selects one they want to see showtime details for
    - 5. System retrieves showtimes and locations for selected movies, sorted by day
    - 6. Customers sees available locations and showtimes for the current day, with ability to scroll through following days
- Entry Conditions:
    - 1. System is connected to theater API
    - 2. Customer is within feasible distance to any location
    - 3. Computing requirements: Compatible browser/app, Internet connection for live updates

3.3.2 Use Case #3

- **Name of Use Case:** Ticket Data Visibility Pull
- Actor(s): SystemAdministrator
- Flow of events:
    - 1. System Administrator connects to ticket selling application backend
    - 2. System checks System Administrator authentication, engages in authentication process if necessary
    - 3. System fetches necessary data for backend management (Ticket data, performance tests, etc)
    - 4. System Administrator selects parameters for what data they would like to have presented (Day, Movie, Location, etc)
    - 5. System retrieves data based on system administrator's request
    - 6. System Administrator utilizes retrieved data for data management purposes
- Entry Conditions:
    - 1. System is connected to theater API
    - 2. System Administrator is already authenticated to view backend, or has necessary keys to pass authentication
    - Computing requirements: Compatible access application (browser/tool), Internet connection for active verification and live updates

…

3.4 Classes / Objects

3.4.1 Class - Ticket

3.4.1.1 Attributes

1. **Class: Ticket**

   - Attributes

   a. ticketId : String - Every ticket generated has a unique id, as mentioned in functional requirements 3.2.4
   b. seatNumber : String - Every ticket generated has a unique, non replicable seat number to ensure every ticket represents (1) unique seat position as mentioned in 3.2.2 and 3.2.3
   c. price : Decimal - Every ticket generated has a tagged price to provide data insights to as mentioned in 3.2.2 and 3.2.3

3.4.1.2 Functions

- Functions

a. generateQRCode() : Image - Creates unique scannable QR code for customer usage, as mentioned in 3.2.4
b. issueChainToken() : Void - Creates unique blockchain record as mentioned in 3.2.4

3.4.2 Class - Showtime

3.4.2.1 Attributes

2. **Class: Showtime**

   - Attributes

   a. timeslot : String - every showtime has a specific immutable timeslot that links to corresponding seatmap, for functionality with use case 1 and 2
   b. showingType : String - every showtime has an attribute noting what type of showing it is (imax, standard, etc) for functionality with use case 1 and 2
   c. availableSeats : Integer - attribute noting how many remaining seats are in a specific showtime for functionality with use case 2 and 1

3.4.2.2 Functions

- Functions

a. lockSeats(seatList : List) : Boolean - Locks selected seats for 5 minutes for usage in use case 1
b. releaseSeats(seatList : List) : Void - if timer runs out or payment fails, seats are unlocked , for usage in use case 1

3.4.3 Class - Location

3.4.3.1 Attributes

3. **Class: Location**

- Attributes

a. coordinate : decimal - every location in the system has a set of coordinates used for user distance determinations, for functionality with use case 2 and 1
b. locationName : string - every location has a name, provided by the theater (typically just the theater name) as mentioned in 3.2.1
c. showings : String - every location has attached mutable showings listed, for use case 2 and 1
d. showtimeID : integer - every showtime has a unique ID for overlap protection for use case 2 and 1

3.4.3.2 Functions

- Functions

a. getMovies() : List - retrieves movie list from theater API for usage in use case 2 and 1
b. updateSchedule() : void - Retrieves theater scheduling from theater API for usage in use case 2 and 1

3.4.4 Class - User

3.4.4.1 Attributes

4. **Class: User**

- Attributes

a. userType : string - every user has a type assigned to them to determine purpose(customer, theater staff, system administrator) for use case 3 and future related use cases as
b. userID : integer - every user has a unique user ID for task handling and overlap protection for use case 3 and future related use cases

3.4.4.2 Functions

- Functions

a. authenticate(credentials : String) : Boolean - validates user has necessary authorization for their desired access point, for usage in use case 3 and future related cases
b. fetchData(params : Query) : Data - administrator data visibility returned based on their inputted parameters, for usage in use case 3

3.5 Non-Functional Requirements

3.5.1 Performance

- 90% of ticket transactions must be processed in less than 5 seconds to prevent user frustration and curate positive user interaction experience.

3.5.2 Reliability

- The system must sustain a Mean Time Between Failures (MTBF) of 40 days minimum

3.5.3 Availability

- System downtime must not exceed 3 minutes per day to prevent users from deciding to not make a purchase they had previously decided on making. Lengthened downtimes for system maintenance must not be during high traffic periods, should strictly be planned between the hours of (1:30am - 5:30am), only being extended outside of that range if necessary and unavoidable

3.5.4 Security (Information referenced from https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/ and https://www.clickssl.net/blog/tls-versions)

- All information exchanges between digital storefront, APIs, and physical kiosks must use TLS 1.2 encryption or better

3.5.5 Maintainability

- System must be capable of integrating a new theaters API information no later than 5 business days after obtaining necessary documentation

3.5.6 Portability

- The digital storefront should be fully functional across main browsers (eg. chrome and safari) and should maintain similar functionality between desktop browser access and mobile browser access.

^^^ We gotta add that stuff below here I think

GITHUB LINK:

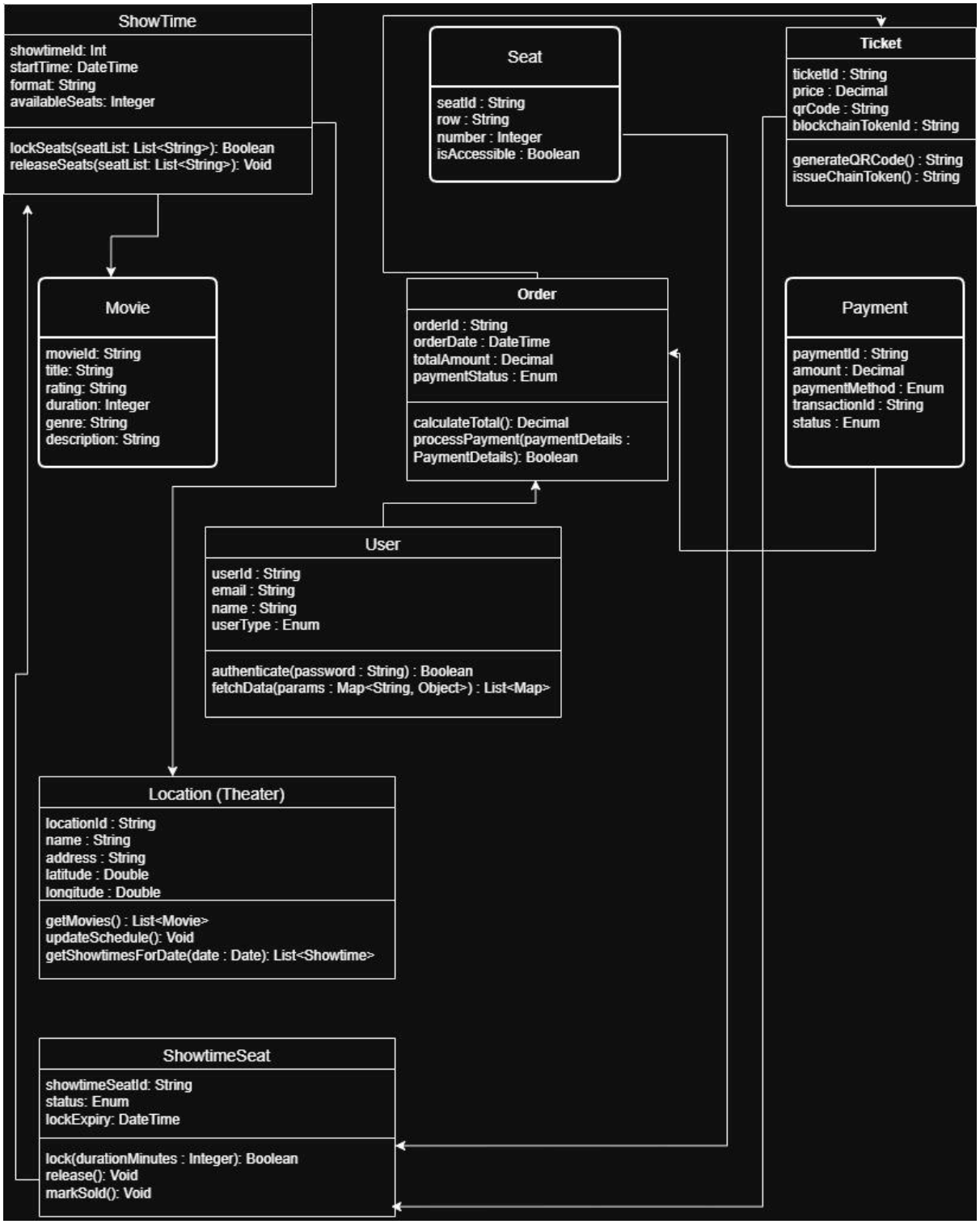
Other Information:

Overview of the Software System:

1. The software system supports the functionality of multi interface user purchases, theater data maintenance, and company data management. It allows for users to browse movies by locations and showtimes and purchase tickets if they wish. Every theater API must be updated with relevant showtime, movie format, and prices so that the software can accurately reflect it on every interface (Physical Kiosk, Mobile App, Website). The company data management must correctly manage theater data for unique locations and securely and accurately store user purchase/ticket information.

2. Tasks ->
    - Clarity and Writing (Andrew - 10 minutes)
    - Create UML Class Diagram (Gershom - 30 minutes)
    - Create UML Class Description (Gershom - 30 minutes)
    - Create SWA Diagram (Jack/Andrew - 45 minutes)
    - Create SWA Description (Jack/Andrew - 45 minutes)
    - Other Information (Jack - 10 minutes)
3. Timeline

- UML Class Diagram (30 minutes) -> UML Class Description (30 minutes) -> SWA Diagram (45 minutes) -> SWA Description (45 minutes) -> Other Information (10 minutes) -> Clarity and Writing (10 minutes)



This UML Class Diagram outlines the blueprint for the TPM System. It’s designed to handle everything from the physical layout of a cinema to the digital "locking" of seats during a high-stakes ticket purchase.

1. Core Content Classes

These classes represent the "static" data of the system, such as the movies and the physical locations where they play.

a) Movie: Represents the cinematic content available in the system.
   Attributes: movieId (String), title (String), rating (String), duration (Integer), genre (String), description (String).
   Operations: None since it's a standard data container.
b) Location (Theater): Represents a physical cinema branch.
   Attributes: locationId (String), name (String), address (String), latitude (Double), longitude (Double).
   Operations:
   - getMovies(): Returns List<Movie>.
   - updateSchedule(): Refreshes showtimes from an external API.
   - getShowtimesForDate(date): Returns List<Showtime>.

2. Scheduling & Inventory

This is the engine of the diagram, connecting movies to physical spaces and managing seat availability.

a) **Showtime**: A specific instance of a movie screening at a specific location.
   <u>Attributes</u>: showtimeId (String), dateTime (DateTime), showingType (Enum), availableSeats (Integer).
   <u>Operations</u>:
   - lockSeats(seatIds, userId): Reserves seats for 5 minutes.
   - releaseSeats(seatIds): Clears locks if payment fails.
   - getAvailableSeats(): Returns List<Seat>.

b) **Seat**: The physical chair in the theater.
   <u>Attributes</u>: seatId (String), row (String), number (Integer), isAccessible (Boolean).
c) **ShowtimeSeat** (Association Class): One of the most critical logic pieces that maps a Seat to a Showtime to track its state for that specific time slot.
   <u>Attributes</u>: showtimeSeatId (String), status (Enum: AVAILABLE, LOCKED, SOLD), lockExpiry (DateTime).
   <u>Operations</u>: lock(duration), release(), markSold().

3. Transactional & User Entities

These classes handle the money, the ownership, and the people involved.

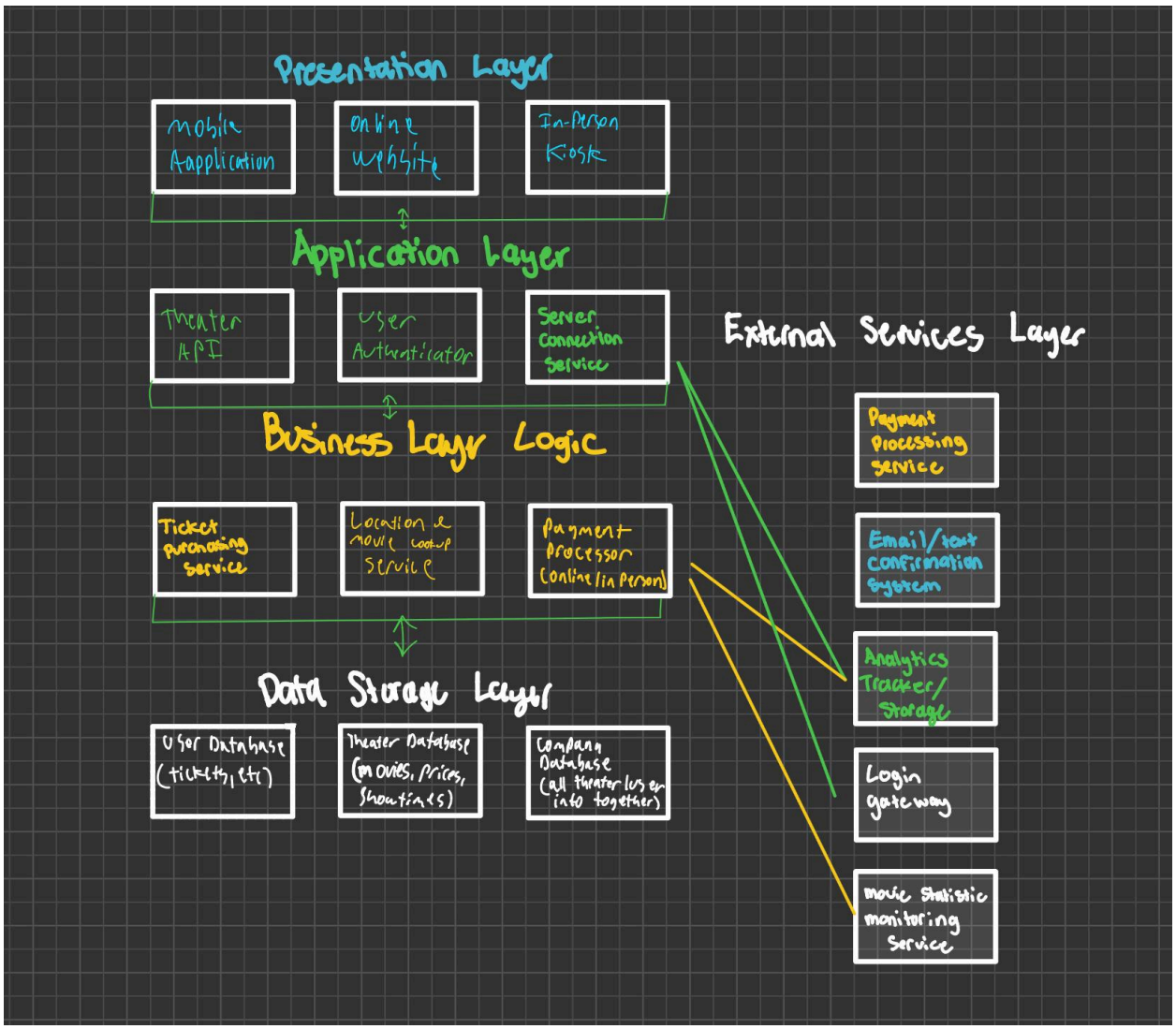
a) **Ticket**: The digital proof of purchase for a specific seat.
   <u>Attributes</u>: ticketId (String), price (Decimal), qrCode (String), blockchainTokenId (String).
   <u>Operations</u>: generateQRCode(), issueChainToken().
b) **Order**: A wrapper for one or more tickets purchased by a user.

Attributes: orderId (String), orderDate (DateTime), totalAmount (Decimal), paymentStatus (Enum).
Operations: calculateTotal(), processPayment(details).

c) **Payment**: The financial record of the transaction.
Attributes: paymentId (String), amount (Decimal), paymentMethod (Enum), transactionId (String), status (Enum).

d) **User**: The person interacting with the system (Customer, Staff, or Admin).
Attributes: userId (String), email (String), name (String), userType (Enum).
Operations: authenticate(password), fetchData(params).

SWA DIAGRAM:



SWA DESCRIPTION:

OVERALL SUMMARY

This system supports movie ticket browsing and purchases through multiple different interfaces (mobile app, website, and in-person kiosk). It uses a layered architecture for the diagram including a presentation layer, Application layer, Business layer, and Data Storage layer. It also includes an external service layer for third party functions like payments, notifications, analytics, and authentification. Requests for purchases start from the user interface in the presentation layer into the application layer, which gets logged into the Business layer for finalizing the purchase, and finally into the Data Storage layer to keep track of the movie purchased, seat selection, and business logistics.

The presentation layer handles user interfaces, and is made up of the mobile application, the online website, and the in person kiosks. It connects to the application layer and business layer, sending user purchase information downstream.

The application layer is made up of the theater API, user authenticator, and server connection service. It handles the threat database, user authentication, and server connection services. These enable the business layer logic to function properly by ensuring accurate movie data, payment data, and ticket information gets passed downstream.

The business layer logic allows users to purchase tickets with all necessary information from upstream, allows users to look up location and showtime data, and handles payment processing to ensure successful transactions. It passes data about user ticket purchases, theater movie/showtime data, and payment data downstream to the data storage layer. In the

Data Storage Layer, data is moved across the designated databases (User database, theater database, ticket/order database). Business services read these inputs to retrieve showtimes, manage seat availability, and record completed purchases and which movie it was completed under.

The external services Layer is called whenever it is needed from the main layers. Payment processing service handles online card transactions. Email and text confirmations of the customers receipts are sent once the purchase is finalized and confirmed. Analytics tracker logs the purchased events. Login Gateway provides external identity. Finally, Movie Statistics Monitoring collects usage and purchase statistics.